



**Tecnológico
de Monterrey**

07. APIs

Ciencia de datos II

**Jorge Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Librerías a descargar

- * httr
- * jsonlite
- * inegiR
- * tidyquant
- * tuber
- * openmeteo

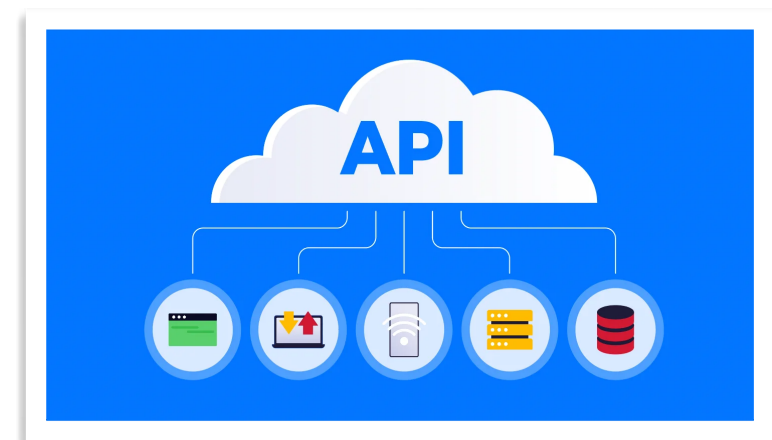
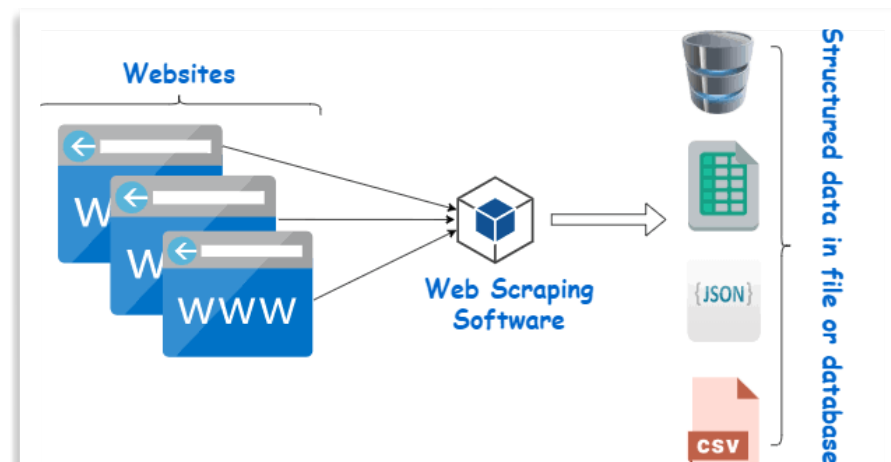


Descargar también **Ollama** y los modelos **Qwen** y **Llama**

Fuentes de los datos

Los datos que utilizamos pueden provenir de varias fuentes:

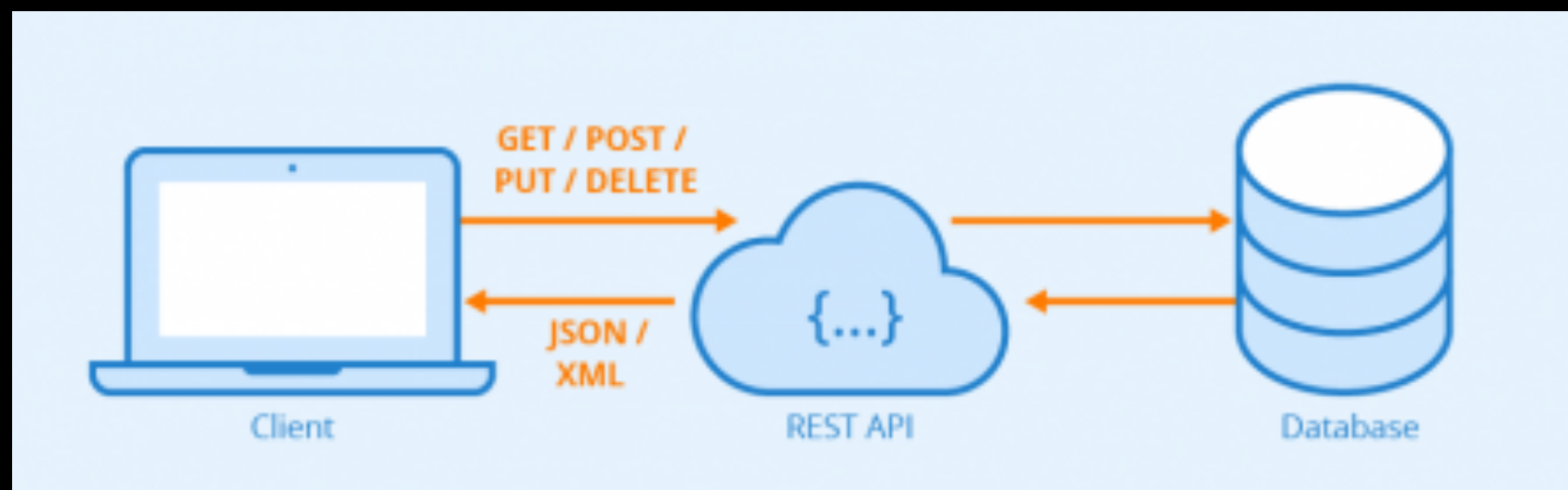
- **Datos abiertos y archivos**
- **Información de la web**
- **APIs**



APIs

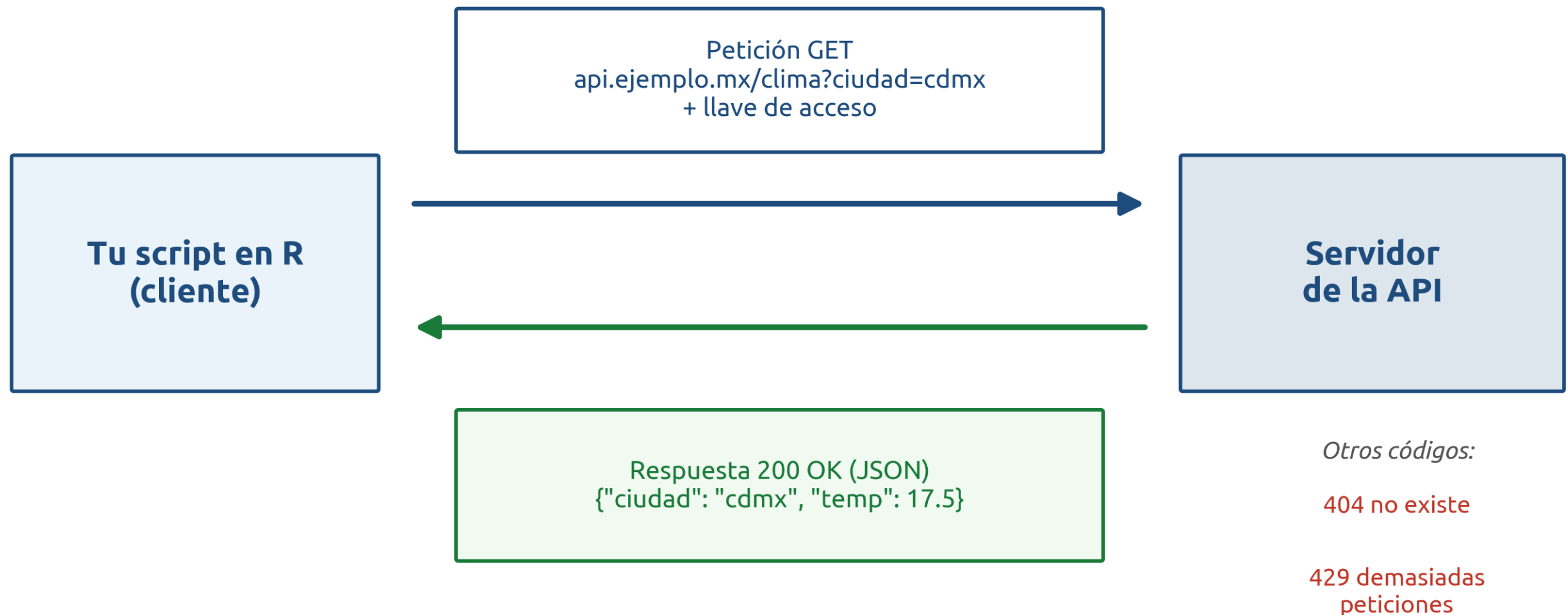
API (Application Programming Interface o Interfaz de Programación de Aplicaciones).

Las APIs son **componentes** del servidor que **hacen que sea fácil que nuestro código interactúe con un servicio y obtenga datos de este.**



Funcionamiento de un API REST

Una API REST es un como un **contrato**: **pides** con una **URL**, **recibes** un **JSON**:



★ API = interfaz de programación de aplicaciones

Es una página web para máquinas: en lugar de HTML para humanos, devuelve datos estructurados para código. El contrato dice qué puedes pedir y qué te responden.

★ REST: Representational State Transfer (Transferencia de Estado Representacional)

Interfaz donde interactúas con recursos mediante URLs y verbos HTTP, siguiendo estos principios: Cliente-servidor, Sin estado; Recursos identificables por URL; Uso de métodos HTTP estándar, Representaciones en JSON o XML, Cacheable.

APIs en R



En una contexto de R:

- Las APIs **son como páginas web, pero para máquinas.**
- Pueden ser utilizadas para **importar grandes cantidades de datos** de manera automática.
- Nos permiten hacer **búsquedas para pedazos específicos de los datos**, dentro de un universo de datos muy grande.

APIs en R



– Para R, existen dos formas de usar las APIs.

Con la librería `{httr}`

La librería `httr` nos permite hacer consultas a las APIs de una manera tradicional (muy similar a como lo haría un programador).

A través de API Clients

Son librerías de R específicas para acceder a las APIs de diferentes servicios. Ejemplos: `{spotifyr}`, `{rtweet}`, `{tuber}`, o `{pageviews}`.

Cientes APIs para R

La forma fácil

- Siempre usar un cliente, cuando sea posible.
- Son interfaces nativas para acceder a las APIs con código y funciones de R. (Son como cualquier otra librería).
- Ocultan completamente la API, y nos permite acceder solo a los resultados estructurados.
- Nos permiten leer los datos a través de objetos de R, en vez de tener que lidiar con archivos JSON.

Etiqueta al usar APIs



– Muchas APIs nos exigen un registro previo, esto con el fin de **controlar el acceso y el uso moderado.**

– Para usar una API, tras el registro, se nos dan tokens de acceso para controlar el acceso y el consumo de los servicios. **Estas llaves muchas veces van incluidas en nuestro código.**

Etiqueta al usar APIs



- Sobrecargar una API con solicitudes puede **reducir la calidad del servicio** tanto para ti como para otros usuarios.
- Muchas APIs regulan el consumo de datos: **permitiendo cierto numero de consultas durante cierto tiempo.**

¿Y si no hay cliente de APIs?



Cuando no hay un paquete intermediario que nos permita comunicarnos con un API, hay que **conectarnos de manera directa con este servicio.**

Para esto, hay que utilizar las librerías *{httr}* y *{jsonlite}*

Otros clientes API para R

Para APIs no-LLM, usa el cliente dedicado cuando exista:

`{tuber}`

YouTube Data API: videos, canales, comentarios.

`{inegiR}`

INEGI: PIB, ENOE, ENIGH, indicadores del BIE.

`{tidyquant}`

Precios de acciones, Yahoo Finance, tipos de cambio.

`{spotifyr}`

Spotify: playlists, popularidad, audio features.

`{openmeteo}`

Clima pronosticado e histórico (sin token).

Siempre busca el cliente primero; solo cae a `httr` si no existe.

Solicitudes HTTP:

Solicitudes HTTP

- Una conversación entre tu máquina y el servidor
- Primero: qué quieres que suceda
- 'métodos' - diferentes solicitudes para diferentes tareas

GET y POST

- **GET:** 'dame algo' ("*El bueno*")
- **POST:** 'toma algo mío'

Otros tipos

- **HEAD** - igual que head() de R: muestra primeros registros
- **DELETE** - 'elimina esto'
- Hay muchos otros, pero GET y POST son los más importantes

Ejercicio práctico 1



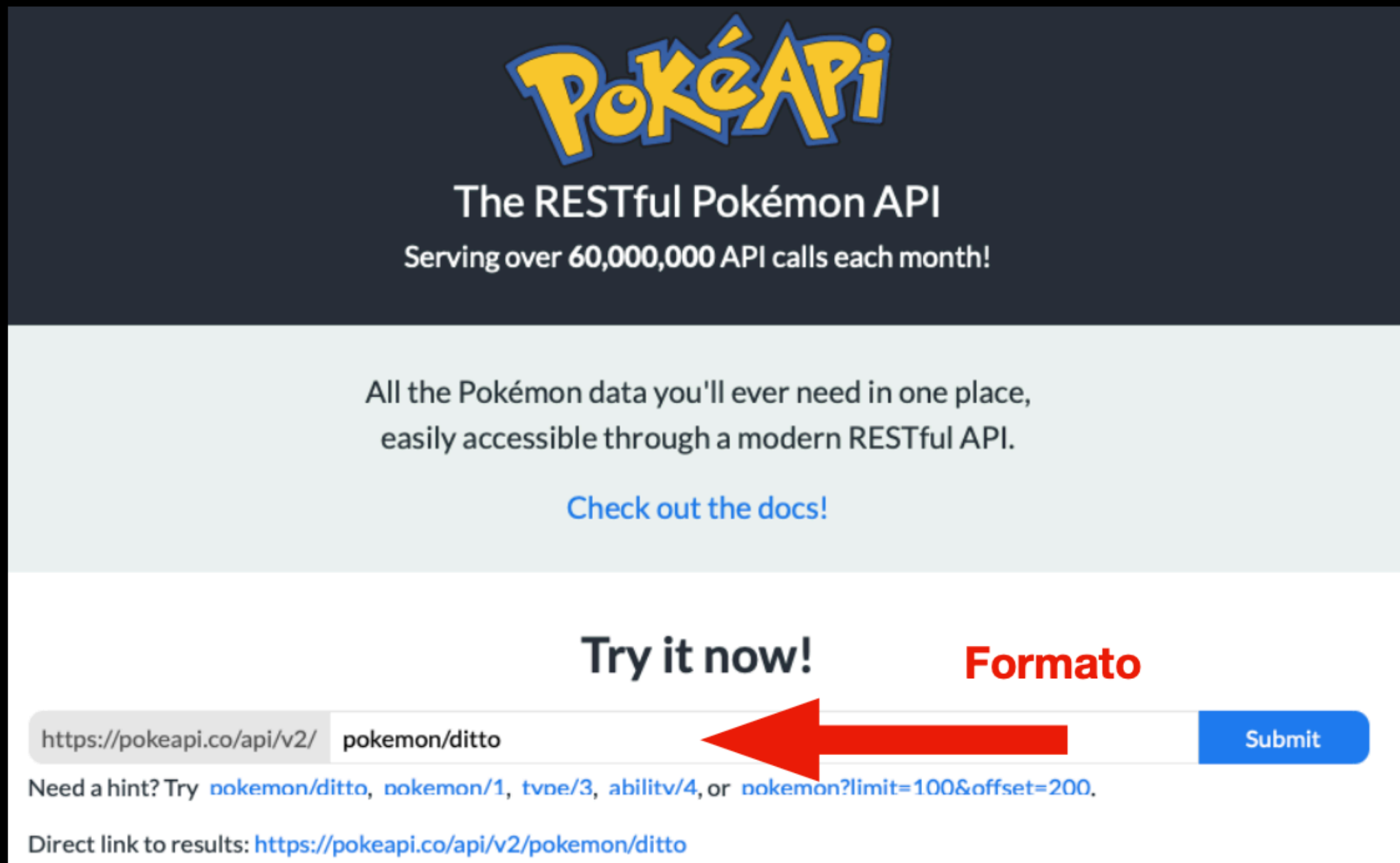
Vamos a ir a la siguiente página:

<https://pokeapi.co/>

Siga las instrucciones de las diapositivas que siguen.

Haciendo una conexión directa

Paso 0. Construyo mi petición en el formato que me lo indique la documentación.



The screenshot shows the PokéAPI website. At the top, the logo "PokéAPI" is displayed in a stylized yellow font with a blue outline. Below it, the text "The RESTful Pokémon API" and "Serving over 60,000,000 API calls each month!" are shown. A light blue section contains the text "All the Pokémon data you'll ever need in one place, easily accessible through a modern RESTful API." and a link "Check out the docs!". The bottom section features a "Try it now!" button and a "Formato" label. Below these is a search bar with the URL "https://pokeapi.co/api/v2/pokemon/ditto" and a "Submit" button. A large red arrow points from the "Formato" label to the search bar. Below the search bar, there is a hint: "Need a hint? Try [pokemon/ditto](#), [pokemon/1](#), [type/3](#), [ability/4](#), or [pokemon?limit=100&offset=200](#)." and a direct link to results: "Direct link to results: <https://pokeapi.co/api/v2/pokemon/ditto>".

PokéAPI
The RESTful Pokémon API
Serving over 60,000,000 API calls each month!

All the Pokémon data you'll ever need in one place,
easily accessible through a modern RESTful API.

[Check out the docs!](#)

Try it now! **Formato**

Need a hint? Try [pokemon/ditto](#), [pokemon/1](#), [type/3](#), [ability/4](#), or [pokemon?limit=100&offset=200](#).

Direct link to results: <https://pokeapi.co/api/v2/pokemon/ditto>

Haciendo una conexión directa



Paso 1. Ya que vi como se tenía que construir la pregunta (llamada), la construyo.

parte_constante + parte_variable

```
# Llamada a la API  
call1 <- paste0("https://pokeapi.co/api/v2/",  
               "pokemon/pikachu/")
```


Haciendo una conexión directa



Paso 2. Dada la dirección de la API (la llamada), hago una GET request, con la función *httr::GET()*, lo cual nos devuelve una respuesta de la información solicitada.

```
# Realizamos la llamada (Obtenemos la respuesta)
llamada <- GET(call1)
llamada # Status 200: Todo ok.
class(llamada) # Tipo Response
```

Objeto response



```
> llamada
Response [https://pokeapi.co/api/v2/pokemon/pikachu/]
Date: 2020-10-01 18:35
Status: 200
Content-Type: application/json; charset=utf-8
Size: 191 kB
```

Códigos de estado HTTP

Toda respuesta de API trae un **status code**. No solo 200.

200

OK — todo bien.

401

Unauthorized — falta el token o está mal.

403

Forbidden — tu token no tiene permisos.

404

Not Found — el recurso no existe.

429

Too Many Requests — excediste el rate limit.

500

Server Error — falló el servidor, reintenta después.

Siempre verifica con `http::status_code(respuesta)` antes de procesar.

Haciendo una conexión directa



Paso 3. De mi objeto response, aplico la función *httr::content()* para acceder al contenido.

```
# De la llamada, obtenemos la respuesta en formato JSON
```

```
# Formato JSON: https://en.wikipedia.org/wiki/JSON
```

```
get_data <- content(llamada, "text")
```

```
get_data
```

```
class(get_data)
```

Datos tipo JSON



```
\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\", \"version_group\": {\"name\": \"crystal\", \"url\": \"https://pokeapi.co/api/v2/version-group/4/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"tutor\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/3/\"}}, {\"version_group\": {\"name\": \"emerald\", \"url\": \"https://pokeapi.co/api/v2/version-group/6/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}}, {\"version_group\": {\"name\": \"diamond-pearl\", \"url\": \"https://pokeapi.co/api/v2/version-group/8/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}}, {\"version_group\": {\"name\": \"platinum\", \"url\": \"https://pokeapi.co/api/v2/version-group/9/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}}, {\"version_group\": {\"name\": \"heartgold-soulsilver\", \"url\": \"https://pokeapi.co/api/v2/version-group/10/\"}}] [\"move\": {\"na
```

Haciendo una conexión directa



Paso 4. Una vez que tengo mi objeto respuesta arrojó el resultado en formato *JSON*, lo convierto en un objeto tipo *list* con la función *jsonlite::fromJSON()*

```
# Del JSON, convertimos este texto en un objeto lista
get_data_JSON <- fromJSON(get_data, flatten = TRUE)
class(get_data_JSON)
```

Objeto ordenado en una lista



```
• get_data_JSON List of 17
  abilities : 'data.frame': 2 obs. of 4 variables:
    ..$ is_hidden : logi [1:2] FALSE TRUE
    ..$ slot : int [1:2] 1 3
    ..$ ability.name: chr [1:2] "static" "lightning-r...
    ..$ ability.url : chr [1:2] "https://pokeapi.co/a...
  base_experience : int 112
  forms : 'data.frame': 1 obs. of 2 variables:
    ..$ name: chr "pikachu"
    ..$ url : chr "https://pokeapi.co/api/v2/pokemon-...
  game_indices : 'data.frame': 20 obs. of 3 variable...
    ..$ game_index : int [1:20] 84 84 84 25 25 25 25 ...
    ..$ version.name: chr [1:20] "red" "blue" "yellow...
    ..$ version.url : chr [1:20] "https://pokeapi.co/...
  height : int 4
  held_items : 'data.frame': 2 obs. of 3 variables:
    ..$ version_details: List of 2
    .. ..$ : 'data.frame': 10 obs. of 3 variables:
```

Cosas a tomar en cuenta de las APIs

- * **A veces, las API cuestan** 🤑, ya que son parte del negocio de las empresas.
- * El uso de la mayoría de las APIs **implica registrarse a sus servicios**, para obtener un *token* de acceso.
- * Las APIs **tienen guías de uso** y documentación que hay que leer para saber como usar
- * Se recomienda **guardar los tokens de acceso** con la funcion *renviron::use_this()*.
- * **Estos tokens de acceso no se comparten ni se suben a Github.** Hasta la IA los regaña si lo hacen.



Ejercicio práctico 2:

- * Utilice la PokeAPI y genere una tabla con el peso y la altura de los primeros 151 Pokemon, incluyendo liga a los *sprites* de los juegos.
- * Utilice el API de Open Meteo API y obtenga el clima pronosticado para esta semana para las coordenadas del Tec de Monterrey Campus Ciudad de México.
- * Utilice el API del INEGI para determinar que entidad de la república tiene la mayor aportación al PIB nacional

Tarea:

- * Consiga sus credenciales de la API de Youtube.
- * Con la API de Youtube y el paquete *{tuber}*, descargue los comentarios de algun video relacionado con la polémica de la situación actual en Sinaloa
- * Con la API de Gemini y el paquete *{ellmer}*, determine si los comentarios son ofensivos.
- * Con Ollama, genere una consulta interna para realizar el mismo proceso.

Conceptos a tomar en cuenta:

Loops o bucles en R con *for*

Suponga que quiere hacer un *loop* en R

Paso 1. Definir qué quiero hacer (y qué resultado esperar).

(Generalmente hacemos loops para automatizar tareas repetitivas)

Paso 2. Hago ese proceso para un caso $n = 1$.

(Programo una vez ese proceso para un caso particular)

Paso 3. Identifico las partes variables del proceso.

(¿Qué cambiaría en el caso $n=2, 3, 4, \dots N$)

Paso 4. Cambio las partes variables del paso 3 por objetos de R.

(Que irán cambiando en el loop)

Paso 5. Construyo el *for*

(O uso alguna otra función de *loop*)
definiendo secuencia, contador y proceso

Loops o bucles en R con *for*

¿Algo
tronó?



Si

(Muchas veces las cosas “truenan” cuando hacemos un *loop*. Hay que ver que *excepción* surgió cuando lo íbamos corriendo y hay que ajustar en caso de que sea necesario).

Toca revisar el loop y hacer ajustes en sus componentes (regresar al paso 3)

No

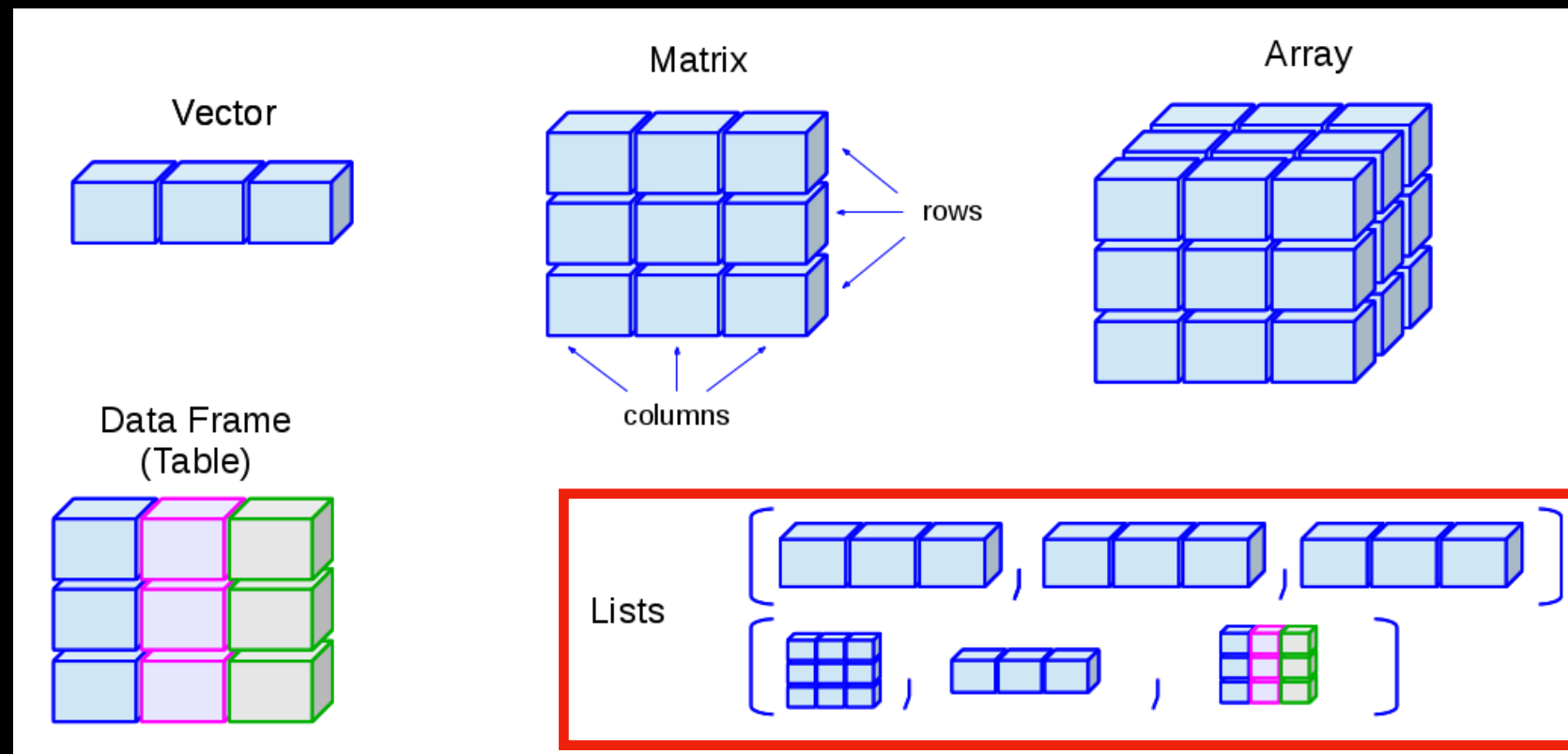


(Que algo no haya “tronado” no significa necesariamente que estemos bien. Hay que verificar si el resultado obtenido es el deseado y, si no, modificar el proceso para corregirlo).



**Si todo está bien: FIN
(Si no, regresar también al paso 3)**

Listas



Una lista es como un vector, en donde el contenido de cada casilla puede ser de **cualquier** tipo (otra lista adentro, un vector, una tibble o dataframe, etc.).

Formato JSON



```
{
  "users": [
    {
      "userId": 1,
      "firstName": "Chris",
      "lastName": "Lee",
      "phoneNumber": "555-555-5555",
      "emailAddress": "clee@fileinfo.com"
    },
    {
      "userId": 2,
      "firstName": "Action",
      "lastName": "Jackson",
      "phoneNumber": "555-555-5556",
      "emailAddress": "ajackson@fileinfo.com"
    }
  ]
}
```

```
"store": {
  "book": [
    {
      "category": "reference",
      "author": "Nigel Rees",
      "title": "Sayings of the Century",
      "price": 8.95
    },
    {
      "category": "fiction",
      "author": "Evelyn Waugh",
      "title": "Sword of Honour",
      "price": 12.99
    },
    {
      "category": "fiction",
      "author": "Herman Melville",
      "title": "Moby Dick",
      "isbn": "0-553-21311-3",
      "price": 8.99
    }
  ]
}
```

Ejemplos:

https://github.com/JuveCampos/Shapes_Resiliencia_CDMX_CIDE/raw/master/Infraestructura/Pozos_Sacmex2.geojson;
Ver archivo carpeta local.

Formato JSON



Un archivo *.json (JavaScript Object Notation) es un **archivo que almacena estructuras simples de datos**.

- Este formato es un **estándar para el intercambio de información**, y se usa para transmitir información entre una aplicación y un servidor.
- Estos archivos son **ligeros, basados en texto, legibles por humanos** y pueden ser editados por un editor de texto.
- Es el **formato** que se utiliza para **dar información proveniente de un API**.
- Para leerlos **en R**, tenemos que usar funciones de la librería **{jsonlite}**.

```
{
  "tag": "select",
  "name": "ctl00$plnMain$ddlClasses",
  "id": "plnMain_ddlClasses",
  "class": "sg-combobox",
  "children": [
    {
      "tag": "option",
      "selected": "selected",
      "value": "ALL",
      "html": "(All Classes)"
    },
    {
      "tag": "option",
      "value": "1319202|1",
      "html": "0020A - 36  STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1319203|1",
      "html": "0020B - 33  STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1232138|1",
      "html": "0113A - 1  APENGLAN A"
    },
    {
      "tag": "option",
      "value": "1232137|1",
      "html": "0113B - 1  APENGLAN B"
    },
    {
      "tag": "option",
      "value": "1229605|1",
      "html": "0284A - 2  PRE CALC PREAP A"
    },
    {
      "tag": "option",
      "value": "1229608|1",
      "html": "0284B - 2  PRE CALC PREAP B"
    },
    {
      "tag": "option",
      "value": "1229088|1",

```

Variables de ambiente

★ **Variable de ambiente:** Son pares clave-valor que el sistema operativo o que R utilizan para almacenar información de configuración.

Funcionan como un contexto que afecta a la ejecución de tu sesión en R, sin necesidad de estar definidas en el script.

Variables de ambiente

Una variable normal existe solo dentro de tu entorno de trabajo.

Una variable de ambiente vive en el sistema y es leída por R. Puede afectar como funcionan los paquetes sin que tengas que ponerlas explícitamente en el código.

Suelen usarse típicamente para guardar llaves de acceso a APIs. Tiene la ventaja de la **seguridad** (evitas poner tus contraseñas en el código) y de **mantenibilidad** (cambias solo una contraseña en vez de cambiar la llave en cada código).

Variables de ambiente

Ejemplo de cómo se vería el archivo .Renviron que carga las variables de ambiente.

Para modificar el archivo .Renviron utilizamos la función `usethis::edit_r_environ()`

```
# Ejemplo en un archivo .Renviron o en el sistema
R_LIBS_USER=/Users/juvenal/R/Library
DB_HOST=database.myserver.com
DB_USER=analyst_user
DB_PASS=SuperSecreta123
API_KEY_YOUTUBE=AIZAaSyAxxxxxxx-xxxxx-xxxx
API_KEY_SPOTIFY=Sp0tifyK3y_123456789
LANG=es_MX.UTF-8
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```


lapply()

Es una función que **nos permite hacer bucles sobre listas**. Tiene la ventaja que no nos exige generar tablas vacías al inicio de un bucle (como lo hacíamos con *for*). Nos devuelve como resultado también una lista.

```
lapply(X, FUN, ...)
```

- **X** : un objeto tipo lista o vector.
- **FUN** : la función que se aplicará a cada elemento.
- **...** : argumentos adicionales que quieras pasarle a la función.

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

X: puede ser 1:100 (del uno al 100), puede ser los nombres de los archivos, lo que sea sobre lo que querramos iterar.

FUN: La función que le queremos aplicar a los valores de **X**. Los argumentos adicionales se le pueden aplicar con ...

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

Más sobre el FUN:

Al igual que en python, en un lapply podemos aplicar funciones anonimas (lambdas) que se van a ejecutar a todos los elementos de **X**.

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Si la lista-resultado de un `lapply()` tiene las mismas dimensiones en todos sus elementos, con la función `do.call(rbind, .)` estas listas se pueden convertir en tablas.

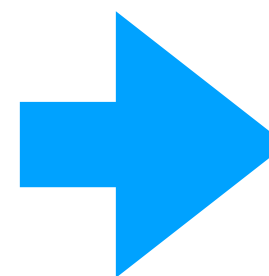
```
library(tibble)

# Tenemos un vector con nombres
personas <- c("Ana", "Luis", "Marta")

# Usamos lapply() para generar una lista de listas
mi_lista <- lapply(seq_along(personas), function(i) {
  tibble(
    nombre = personas[i],
    edad   = 20 + i * 5,
    ciudad = c("CDMX", "Monterrey", "Guadalajara")[i]
  )
})

# Convertimos la lista en tibble con do.call()
mi_tibble <- mi_lista %>% do.call(rbind, .)

mi_tibble
```



nombre	edad	ciudad
<chr>	<dbl>	<chr>
Ana	25	CDMX
Luis	30	Monterrey
Marta	35	Guadalajara

ellmer: cliente unificado

`{ellmer}` es un **cliente unificado para LLMs** hecho por Posit.

- Mismo código para Claude, OpenAI, Gemini, Ollama.
- Maneja autenticación, retry, streaming, tool use.
- Lee las llaves de `.Renviron` automáticamente.
- Soporta **salidas estructuradas** (JSON validado).

```
install.packages("ellmer")  
library(ellmer)
```

chat_google_gemini()

Crear un chat con Gemini es una sola llamada:

```
library(ellmer)

# Asume GOOGLE_API_KEY en .Renviron
chat <- chat_google_gemini(
  model = "gemini-2.0-flash"
)

# Mandar un mensaje y obtener la respuesta
respuesta <- chat$chat("Explicame qué es una API en una oración.")

respuesta
```

El objeto `chat` recuerda el historial por defecto.

Verifica el nombre exacto en `?ellmer` — la API evoluciona rápido.

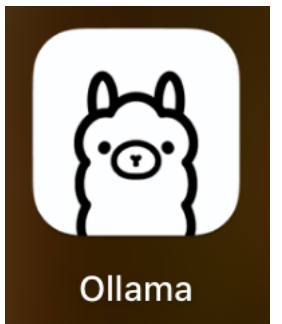
System prompts

Un **system prompt** define el rol y las reglas del modelo antes de empezar.

```
chat <- chat_google_gemini(  
  system_prompt = "  
    Eres un moderador experto en redes sociales.  
    Clasificas mensajes en español como discurso de odio o no.  
    Responde siempre en formato JSON con los campos:  
      - es_discurso_odio (TRUE/FALSE)  
      - razon (string corto)  
  ",  
  model = "gemini-2.0-flash"  
)  
  
chat$chat("Todos los aficionados del otro equipo son unos idiotas.")
```

Buen system prompt = respuestas consistentes y útiles.

Ollama: LLMs locales



Ollama es un servidor que corre LLMs en tu propia máquina.

Ventajas

- **Gratis** (no hay costo por token)
- **Privado** (los datos no salen)
- Sin rate limits
- Funciona sin internet

Desventajas

- Modelos más **pequeños**
- Usa **RAM y GPU**
- Más lento que las APIs de nube
- Calidad menor que GPT-4 o Claude

```
# En terminal, antes de usar desde R:  
ollama pull qwen2.5:7b  
ollama pull llama3.2
```

chat_ollama()

El patrón es idéntico al de Gemini. Solo cambia el constructor:

```
library(ellmer)

chat_local <- chat_ollama(
  system_prompt = "
    Eres un clasificador de discurso de odio y discurso violento.
    Respondes solo TRUE o FALSE si el texto es un texto de odio o violento.
  ",
  model = "qwen3.5:4b"
)

chat_local$chat("Me encantó el partido de ayer.")
# [1] "FALSE"

chat_local$chat("Ojalá se muera ese jugador.")
# [1] "TRUE"
```

Ollama debe estar corriendo (`ollama serve` o la app).

Ejemplo: tuber para YouTube

Flujo típico: autenticación OAuth, luego consulta.

```
library(tuber)
library(tidyverse)

# Autenticación (se abre el navegador la primera vez)
yt_oauth(
  app_id      = Sys.getenv("YT_APP_ID"),
  app_secret  = Sys.getenv("YT_APP_SECRET")
)

# Descargar los comentarios de un video
comentarios <- get_comment_threads(
  filter      = c(video_id = "dQw4w9WgXcQ"),
  max_results = 100
)

comentarios %>%
  as_tibble() %>%
  select(authorDisplayName, textOriginal, likeCount)
```